e-PG Pathshala

Subject: Computer Science

Paper: Operating Systems

Module 4: Process Concepts

Module No: CS/OS/4

Quadrant 1 — e-text

4.1 Introduction

A process is the unit of work in modern time-sharing systems. A system has a collection of processes – user processes as well as system processes. All these processes can execute concurrently with the CPU multiplexed among them. This module describes what processes are, gives an introduction to process scheduling and explains the various operations that can be done on processes.

4.2 Processes

A process is a program in execution. The execution of a process progresses in a sequential fashion. A program is a passive entity while a process is an active entity. A process includes much more than just the program code. A process includes the text section, stack, data section, program counter, register contents and so on. The text section consists of the set of instructions to be executed for the process. The data section contains the values of initialized and uninitialized global variables in the program. The stack is used whenever there is a function call in the program. A layer is pushed into the stack when a function is called. The arguments to the function and the local variables used in the function are put into the layer of the stack. Once the function call returns to the calling program, the layer of the stack is popped. The text, data and stack sections comprise the address space of the process. The program counter has the address of the next instruction to be executed in the process.

It is possible to have two processes associated with the same program. For example, consider an editor program, say Microsoft Word. The program has the same text section. But, the data section will be different for each file that is opened in Microsoft Word, that is, each file has a different data section.

4.3 Process States

As a process executes, it changes state. The state of a process refers to what the process currently does. A process can be in one of the following states during its lifetime:

- new: The process is being created.
- running: Instructions are being executed.
- waiting: The process is waiting for some event to occur.
- ready: The process is waiting to be assigned to a processor.
- terminated: The process has finished execution.

Figure 4.1 shows the state transition diagram of a process. The process is in the new state when it is being created. Then the process is moved to the ready state, where it waits till it is taken for execution. There can be many such processes in the ready state. One of these processes will be selected and will be given the processor, and the selected process moves to the running state. A process, while running, may have to wait for I/O or wait for any other event to take place. That process is now moved to the waiting state. After the event for which the

process was waiting gets completed, the process is moved back to the ready state. Similarly, if the time-slice of a process ends while still running, the process is moved back to the ready state. Once the process completes execution, it moves to the terminated state.

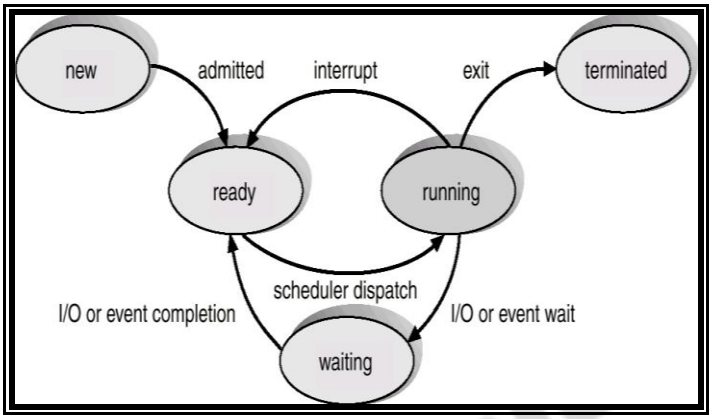

Fig. 4.1 Process state transition diagram (Source: [2])

4.4 Process Control Block (PCB)

The information about each process is maintained in the operating system in a process control block, which is also called a task control block. Figure 4.2 shows a PCB. The PCB contains extensive information about the process. The information present in the PCB includes the following:

- Process state – Current state of the process
- Program counter – Address of the next instruction to be executed
- CPU registers
    - Accumulators, index registers, stack pointer, general purpose registers
- CPU scheduling information
    - Process priority, scheduling parameters
- Memory-management information
    - Value of base and limit registers
- Accounting information
    - Amount of CPU used, time limits, process numbers
- I/O status information
    - List of I/O devices allocated to the process, list of open files and so on.

Pointer	Process state
Process number	
Program counter	
Registers	

Memory limits
List of open files
. . .

Fig. 4.2 Process control block (PCB) (Source: [2])

Figure 4.3 shows how the contents of the PCB of a process are used when the CPU switches from one process to another process. Process P_0 is executing first, and there is an interrupt. Process P_0 should now release the CPU and the CPU should be assigned to the next process P_1. Before the CPU is assigned to the next process P_1, the state of the process currently using the CPU, P_0, is saved in its PCB. The state of the next process P_1 is then loaded from the PCB of P_1. When there needs to be a switch from P_1 to P_0, the state of P_1 is saved and the state of P_0 is loaded.

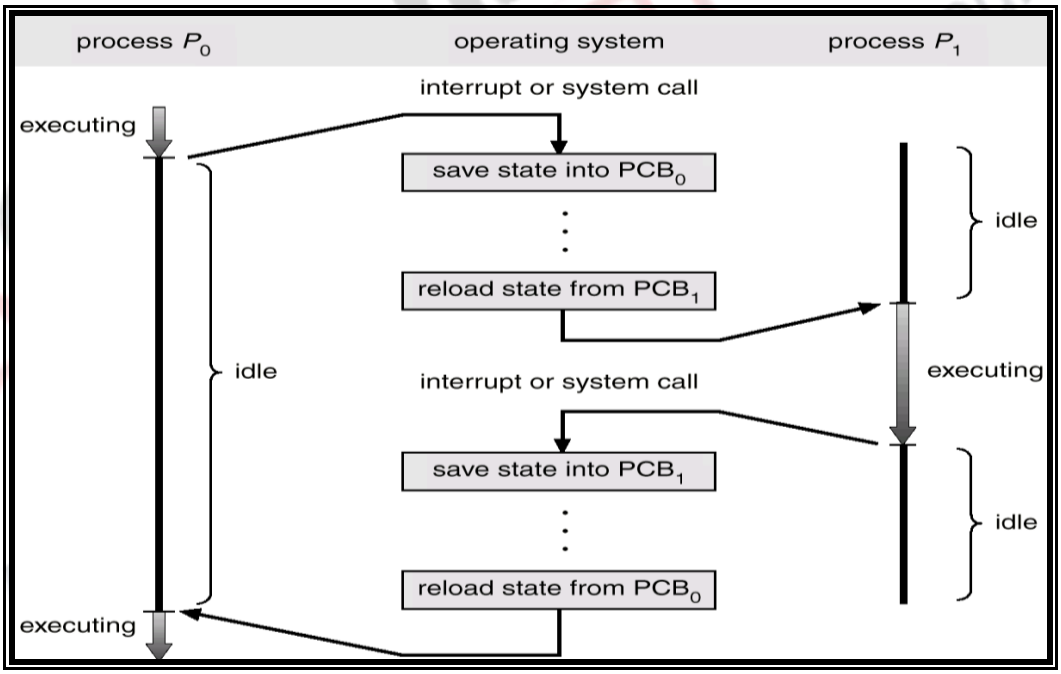

Fig. 4.3 CPU switch from process to process (Source: [2])

4.5 Process Scheduling

In multi-programmed systems, some process must run at all times, to increase CPU utilization. In time-sharing systems, processes must be switched to increase interaction between the user and the system. If there are many processes, only one can use the CPU at a particular time and the remaining must wait during that time. When the CPU becomes free, the next process can be scheduled.

4.5.1 Process Scheduling Queues

When processes enter the system, they are put into a job queue. This job queue is the set of all processes in the system. The set of all processes residing in main memory, ready and waiting to execute is kept in a ready queue. The ready queue is maintained as a linked list. A ready-queue header contains pointers to the first and final PCBs in the list.

In addition to the ready queue, there are other queues in the system in which a process may be kept during its lifetime. When a process has to wait for I/O, the PCB of the process is removed from the ready queue and is placed in a device queue. The device queue corresponds to the I/O device from/to which the process is waiting for I/O. Hence, there are a number of device queues in the system corresponding to the devices present in the system.

Figure 4.4 shows the ready queue and the various device queues in the system. Any process during its lifetime will migrate between the various queues.

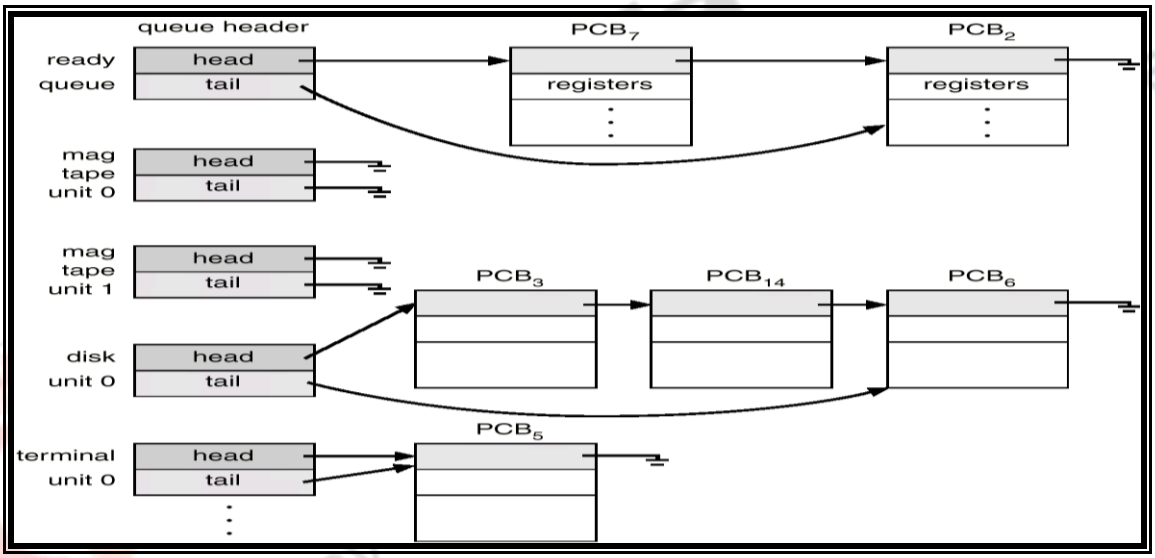

Fig. 4.4 Ready queue and various I/O device queues (Source: [2])

Figure 4.5 shows a common representation of process scheduling using a queuing diagram. The rectangular boxes represent the various queues. The circles denote the resources that serve the queues and the arrows show the flow of processes in the system.

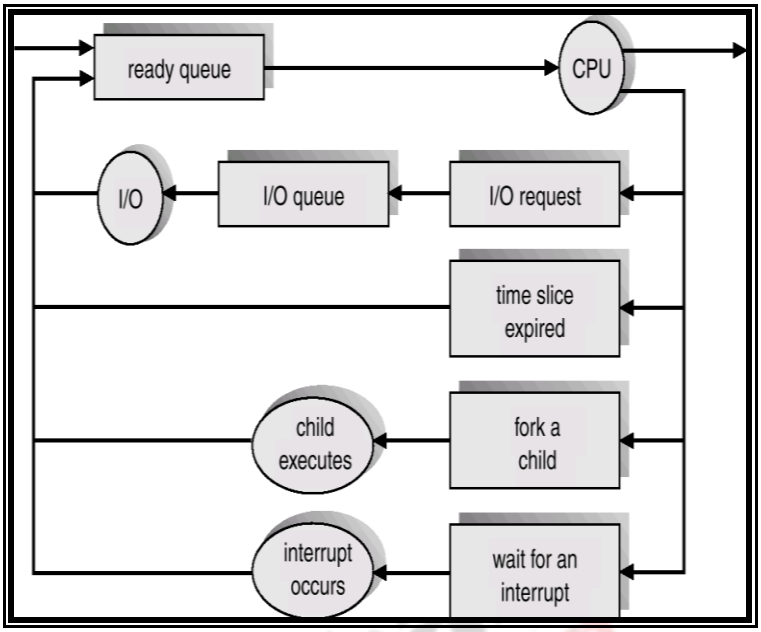

Fig. 4.5 Queuing diagram representation of process scheduling (Source: [1])

A new process is initially placed in the ready queue. When the process is given the CPU and is running one of the following may occur:

- The process may request for I/O and may be placed in an I/O queue. After I/O gets completed, the process is moved back to the ready queue.
- The time slice allotted to the process may get over. The process is now forcibly removed from the CPU and is placed back in the ready queue.
- The process may create (fork) a new process and may wait for the created child process to finish completion. Once the child completes execution, the process moves back to the ready queue.
- While the process executes, an interrupt may occur. The process is now removed from the CPU, the process waits till the interrupt is serviced and is then moved to the ready state.

4.5.2 Schedulers

A process moves between different queues during its lifetime. The OS should select the process that should move to the next queue and the queue to which the selected process should move, in some fashion. This selection is done by schedulers. The different schedulers available are long-term scheduler, short-term scheduler and medium-term scheduler.

In batch systems, more processes are submitted than those that can be executed. These processes are placed in a job pool. The long-term scheduler (or job scheduler) selects those processes that should be brought into the ready queue from the job pool, and brings them from the job pool to main memory. The short-term scheduler (or CPU scheduler), selects the process that be executed next and allocates the CPU.

The main difference between the job scheduler and the CPU scheduler is the frequency of execution. The long-term scheduler controls the degree of multiprogramming (number of

processes in memory). It is invoked only when a process leaves the system. The short-term scheduler is invoked whenever the CPU switches from one process to another. Hence, the short-term scheduler is run more frequently than the long-term scheduler.

As the long-term scheduler controls the degree of multiprogramming, it should select a good mix of I/O-bound and CPU-bound processes and bring them into the main memory. An I/O-bound process spends most of its time performing I/O than computation. A CPU-bound process spends most of its time performing computations than I/O. If all processes are I/O-bound, then the CPU will be under-utilized. If all processes are CPU-bound, the I/O devices will not be used fully. Hence, proper selection of jobs by the job scheduler will ensure that the system is stable.

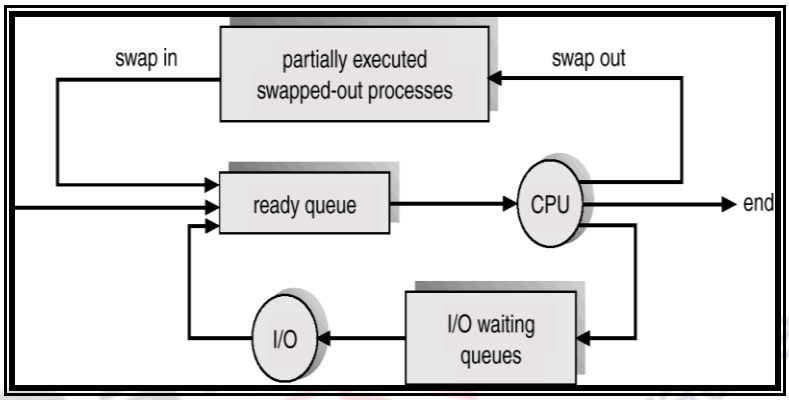

Fig. 4.6 Medium-term scheduling (Source: [2])

Some operating systems, like the time-sharing systems introduce an additional, intermediate level of scheduling, called medium-term scheduling. Any process that is in the running state has to be kept in the main memory. Such a process may not be in the running state throughout its life time till its termination. It may be moving to the waiting state or to the ready queue and then may move back to the running state and so on. Moreover, when one process moves to the waiting state or to the ready queue, another process moves to the running state. The process that is currently in the running state should be kept in the main memory. If there are many processes are in the waiting state or in the ready queue and if all the processes are still kept in the main memory, the main memory may not be enough to accommodate the currently running process. When there is no enough space in the main memory, the medium-term scheduler removes these partially executed processes from the main memory and moves to the swap space in the secondary storage device. Later, when there is space in the main memory, the processes can be brought into the memory and will continue where they left off. This process is also known as swapping. Figure 4.6 shows how a medium-term scheduler works.

4.5.3 Context of a Process

The context of a process is represented in the PCB of the process. It includes the values in CPU registers, process state, memory management information, open files for the process and so on. When the CPU switches from one process to another process, it is said that a context switch occurs. During a context switch, the system must save the context of the old process and load the saved context for the new process. Context-switch time is overhead,

because the system does no useful work while switching.

Context-switch time depends on the memory speed, number of registers to be copied and the existence of special instructions. It is also dependent on hardware support. For example, some processors provide multiple sets of registers. Then, the context for each process can be stored in each set of registers. In this case, context switch will only be moving a pointer from one register set to another.

4.6 Operations on Processes

The operating system must provide support for creation and termination of processes.

4.6.1 Process Creation

Any process can create other processes. The process that creates is called the parent process. The parent process creates children processes, which, in turn create other processes, forming a tree of processes. Each process is identified by a process identifier (pid). In UNIX systems, the ps command is used to list all processes in the system along with their pids.

Any process will need resources such as CPU time, memory and other resources to complete its task. When a parent process creates children processes, the children may obtain resources from the system or may share the resources with the parent. Thus, in some operating systems, the parent and children share all resources. In some other operating systems, the children share a subset of the parent's resources and in some others, the parent and child share no resources.

Once a child process is created, the parent and child execute concurrently. In some cases, the parent does not wait for the child to terminate. In some others, the parent waits until some or all the children terminate. There are again two possibilities about the address space (text, data and stack) of the newly created process. The address space of the child process may be the duplicate of that of the parent process. Instead, the child may have another program loaded into it.

4.6.2. Process creation in UNIX

The fork system call is used to create a new process in UNIX. The address space of the child process is a copy of the address space of the parent process. If the child has to execute some program other than the program that the parent executes, the exec system call is used. The exec system call, when used after a fork, replaces the child process' memory space with a new program. Figure 4.7 shows the flow of the parent and the child processes after a fork is being called by the parent. The parent after creating the child may even wait for the child to exit using the wait system call. In UNIX, the parent can even choose to exit without waiting for the child to exit.

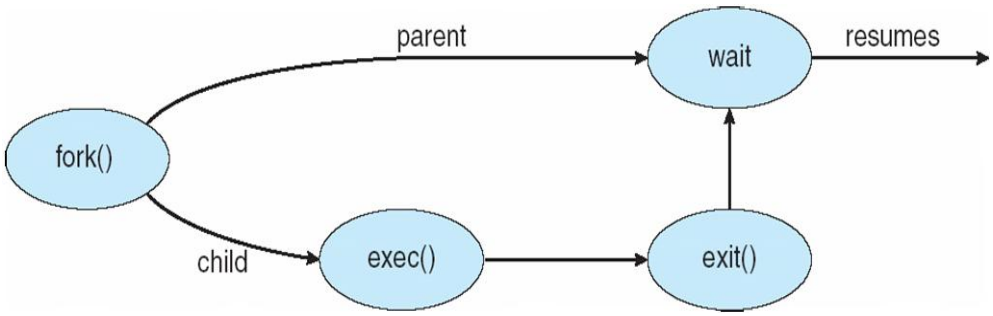

Fig. 4.7 Process creation in UNIX

4.7 Summary

A process is an active entity while a program is a passive entity. A process can be in different states during its lifetime. The context of a process is represented by a process control block. Appropriate schedulers move processes between different states. Operations like creation and termination can be performed on a process.

References

[1] http://www.wiley.com/college/silberschatz6e/0471417432/slides/slides.html
[2] Abraham Silberschatz, Peter B. Galvin, Greg Gagne, “Operating System Concepts”, Sixth Edition, John Wiley & Sons Inc., 2003.

e-Pathshala पाठशाला A Gateway to All Post Graduate Courses